PRINCIPLES OF CRYPTOGRAPHY (POC) - FINAL EXAM CHEAT SHEET

Focus: Topics After DES (Post-Midterm)

1. ADVANCED ENCRYPTION STANDARD (AES)

Key Features

- **Block Size**: 128 bits
- **Key Sizes**: 128, 192, or 256 bits
- **Rounds**: 10 (AES-128), 12 (AES-192), 14 (AES-256)
- **Structure**: Substitution-Permutation Network (not Feistel)

AES Operations

1. **SubBytes**: Non-linear substitution using S-box
2. **ShiftRows**: Circular shift of rows
3. **MixColumns**: Matrix multiplication in $GF(2^8)$
4. **AddRoundKey**: XOR with round key

Advantages over DES

- More secure (larger key space)
- More efficient in software/hardware
- Simpler algorithmic structure
- Flexible key sizes

2. MULTIPLE ENCRYPTIONS

Double DES

- **Encryption**: $C = E_{K2}(E_{K1}(P))$
- **Key Space**: 2^{112}
- **Vulnerability**: Meet-in-the-Middle (MIM) attack reduces to 2^{57}

Triple DES (3DES)

With 2 Keys: $C = E_{K1}(D_{K2}(E_{K1}(P)))$

With 3 Keys: $C = E_{K3}(D_{K2}(E_{K1}(P)))$

- More secure than Double DES
- Prevents MIM attack effectively

3. BLOCK CIPHER MODES OF OPERATION

Electronic Codebook (ECB)

- **Encryption**: $C_i = E(K, P_i)$
- **Pros**: Simple, parallel processing
- **Cons**: Vulnerable to pattern attacks, poor diffusion

Cipher Block Chaining (CBC)

- **Encryption**: $C_1 = E(K, P_1 \oplus IV)$, $C_i = E(K, P_i \oplus C_{i-1})$
- **Decryption**: $P_1 = D(K, C_1) \oplus IV$, $P_i = D(K, C_i) \oplus C_{i-1}$
- **Pros**: Better diffusion than ECB
- **Cons**: Sequential processing, IV management

Cipher Feedback (CFB)

- **Converts block cipher to stream cipher**
- **Encryption**: $C_j = P_j \oplus MSB_s(E(K, I_j))$
- **Pros**: No padding required
- **Cons**: IV management, limited parallelism

Output Feedback (OFB)

- **Encryption**: $C_j = P_j \oplus O_j$ where $O_j = E(K, I_j)$
- **Pros**: No error propagation, parallel key generation
- **Cons**: Vulnerable to message modification

Counter (CTR)

- **Encryption**: $C_j = P_j \oplus E(K, T_j)$
- **Counter**: $T_j = (T_{j-1} + 1) \mod 2^b$
- **Pros**: Parallel processing, preprocessing, random access
- **Cons**: Nonce management critical

4. XTS-AES MODE (Storage Devices)

Features

- **Purpose**: Sector-based storage encryption
- **Structure**: Tweakable block cipher
- **Encryption**: $C = E_{K1}(P \oplus T) \oplus T$
- **Tweak**: $T = E_{K2}(i) \otimes \alpha^j$ in $GF(2^{128})$

Applications

- Full-disk encryption
- Database encryption
- Secure cloud storage

5. STREAM CIPHERS

RC4

- **Type**: Variable key-size stream cipher
- **Operations**: Byte-oriented
- **Structure**: Key-Scheduling Algorithm (KSA) + Pseudo-Random Generation Algorithm (PRGA)
- **Applications**: SSL/TLS (legacy), WEP, WPA

Key Steps

1. **Initialize S and T**: $S[i] = i$, $T[i] = K[i \mod keylen]$
2. **Initial Permutation**: Swap based on $j = (j + S[i] + T[i]) \mod 256$
3. **Stream Generation**: Generate keystream bytes

6. PSEUDORANDOM NUMBER GENERATION

PRNG Requirements

- **Randomness**: Uniform distribution
- **Unpredictability**: Forward & backward
- **Seed Security**: Must be unpredictable

PRNG Using Block Cipher (CTR Mode)

```
while len(temp) &lt; requested_bits:
    V = (V + 1) mod 2^b
    output = E(Key, V)
    temp = temp || output
```

Blum Blum Shub Generator (BBSG)

- **Setup**: $n = p \times q$ (p, q are primes)
- **Seed**: $X_0 = s^2 \mod n$
- **Iteration**: $X_i = X_{i-1}^2 \mod n$, $B_i = X_i \mod 2$
- **Security**: Based on difficulty of factoring n

7. PUBLIC KEY CRYPTOGRAPHY (PKC)

RSA Algorithm

Key Generation

1. Select large primes p and q
2. $n = p \times q$
3. $\phi(n) = (p-1)(q-1)$
4. Choose e: $GCD(e, \phi(n)) = 1$
5. Calculate d: $d = MI(e) \mod \phi(n)$

Encryption/Decryption

- **Encryption**: $C = P^e \mod n$
- **Decryption**: $P = C^d \mod n$
- **Proof**: $C^d = (P^e)^d = P^{ed} = P^{k\phi(n)+1} = P \mod n$

Efficient Operations

- **Common e values**: 3, 17, 65537 ($2^{16} + 1$)
- **CRT for decryption**: 4x speedup
  - $V_p = C^d \mod p$
  - $V_q = C^d \mod q$
  - Combine using CRT

8. DIFFIE-HELLMAN KEY EXCHANGE (DHKE)

Protocol

1. **Public Parameters**: Prime q, primitive root α
2. **Alice**: X_A (private), $Y_A = \alpha^{X_A} \mod q$ (public)
3. **Bob**: X_B (private), $Y_B = \alpha^{X_B} \mod q$ (public)
4. **Shared Secret**:
   - Alice: $K = Y_B^{X_A} \mod q$
   - Bob: $K = Y_A^{X_B} \mod q$

Security

- Based on discrete logarithm problem
- **Vulnerable to**: Man-in-the-Middle (MITM) attack

9. HASH FUNCTIONS

Secure Hash Algorithm (SHA)

SHA-512

- **Message Digest**: 512 bits
- **Block Size**: 1024 bits
- **Rounds**: 80
- **Operations**: $Ch(e, f, g)$, $Maj(a, b, c)$, Σ, σ

Properties of Cryptographic Hash

1. **Variable input, fixed output**
2. **One-way**: Hard to find M from h(M)
3. **Collision-resistant**: Hard to find M, M' where h(M) = h(M')
4. **Avalanche effect**: Small change in input → large change in output

10. MESSAGE AUTHENTICATION CODE (MAC)

HMAC (Hash-based MAC)

Formula: $HMAC(K, M) = H[(K \oplus opad)||H[(K \oplus ipad)||M]]$

Parameters

- **ipad**: 0x36 repeated
- **opad**: 0x5C repeated
- **K**: Secret key (padded to block size)

Security

- Depends on underlying hash function
- Provides integrity and authentication

Requirements for MAC

1. Even knowing MAC function, attacker can't forge
2. MAC should appear random
3. Bit changes shouldn't result in same MAC

11. DIGITAL SIGNATURES

Applications of PKC

For Confidentiality

$Y = E_{PU_b}(X), X = D_{PR_b}(Y)$

For Authentication

$Y = E_{PR_a}(X), X = D_{PU_a}(Y)$

For Both

$Z = E_{PU_b}(E_{PR_a}(X))$

$X = D_{PU_a}(D_{PR_b}(Z))$

Hash with Digital Signature

1. Compute hash: $h = H(M)$
2. Sign hash: $S = E_{PR_a}(h)$
3. Send M and S
4. Verify: $h' = D_{PU_a}(S)$, check $h' = H(M)$

12. KEY CONCEPTS FROM PRE-MIDTERM (REVIEW)

Modular Arithmetic

- $a \mod n = r$ where
- **Multiplicative Inverse**: $a \times b \equiv 1 \pmod n$, find using EEA
- **Properties**:
  - $(a + b) \mod n = [(a \mod n) + (b \mod n)] \mod n$
  - $(a \times b) \mod n = [(a \mod n) \times (b \mod n)] \mod n$

Extended Euclidean Algorithm (EEA)

Finds $GCD(a, b)$ and x, y such that $ax + by = GCD(a, b)$

Usage: Find multiplicative inverse $MI(a) \mod n$

Fermat's & Euler's Theorems

- **Fermat**: $a^{p-1} \equiv 1 \pmod p$ if p is prime and $GCD(a, p) = 1$
- **Euler**: $a^{\phi(n)} \equiv 1 \pmod n$ if $GCD(a, n) = 1$
- **Euler Totient**: $\phi(p \times q) = (p - 1)(q - 1)$ for primes p, q

Chinese Remainder Theorem (CRT)

Solves system: $x \equiv a_i \pmod{m_i}$ for pairwise coprime m_i

Formula: $x = \sum_{i=1}^{k} a_i M_i N_i \mod M$

- $M = \prod m_i$
- $M_i = M/m_i$
- $N_i = MI(M_i) \mod m_i$

Discrete Logarithms & Primitive Roots

- **Order**: $Order_n(a)$ = smallest k where $a^k \equiv 1 \pmod{n}$
- **Primitive Root**: α is primitive root of n if $Order_n(\alpha) = \phi(n)$
- **Discrete Log**: If $b = \alpha^x \mod n$, then $x = dlog_{\alpha,n}(b)$

Miller-Rabin Primality Test

Test if n is prime:

1. Write $n - 1 = 2^k \times q$ (q odd)
2. Choose random a
3. Compute $a^q \mod n$
4. If = 1 or n-1, possibly prime
5. Square repeatedly; if get n-1, possibly prime
6. Else composite

IMPORTANT FORMULAS QUICK REFERENCE

Cryptographic Operations

Operation	Formula
RSA Encrypt	$C = P^e \mod n$
RSA Decrypt	$P = C^d \mod n$
DHKE Public Key	$Y = \alpha^X \mod q$
DHKE Shared Secret	$K = Y^X \mod q$
HMAC	$H[(K \oplus opad) \|\| H[(K \oplus ipad) \|\| M]]$

Mode Operations

Mode	Encryption	Decryption
ECB	$C_i = E(K, P_i)$	$P_i = D(K, C_i)$
CBC	$C_i = E(K, P_i \oplus C_{i-1})$	$P_i = D(K, C_i) \oplus C_{i-1}$
CTR	$C_i = P_i \oplus E(K, T_i)$	$P_i = C_i \oplus E(K, T_i)$

COMPARISON TABLES

Symmetric vs Asymmetric Encryption

Feature	Symmetric	Asymmetric
Keys	Same key	Key pair (public/private)
Speed	Fast	Slow
Key Size	128-256 bits	1024-4096 bits
Use	Bulk data	Key exchange, signatures
Examples	AES, DES	RSA, DHKE

Block Cipher Modes

Mode	Parallel Enc	Parallel Dec	Error Prop	Padding	Stream Cipher
ECB	✓	✓	✗	✓	✗
CBC	✗	✓	✓	✓	✗
CFB	✗	✗	Limited	✗	✓
OFB	✓ (prep)	✓ (prep)	✗	✗	✓
CTR	✓	✓	✗	✗	✓

COMMON ATTACKS & COUNTERMEASURES

On Block Ciphers

Attack	Target	Countermeasure
Brute Force	All	Large key size
Meet-in-Middle	Double DES	Use Triple DES
Pattern Analysis	ECB	Use CBC/CTR
IV Manipulation	CBC	Protect IV integrity

On Public Key

Attack	Target	Countermeasure
Factoring	RSA	Large primes
Discrete Log	DHKE	Large prime q
Small e	RSA	Use e=65537
MITM	DHKE	Authentication

On Hash Functions

Attack	Complexity	Countermeasure
Collision	$2^{n/2}$	Larger output
Preimage	2^n	One-way property
Length Extension	Varies	Use HMAC

EXAM STRATEGY

High-Probability Numerical Topics

1. **RSA**: Complete key generation, encryption, decryption
2. **DHKE**: Calculate shared secret from given parameters
3. **Modular Exponentiation**: $a^b \mod n$ using fast exponentiation
4. **EEA**: Find multiplicative inverse
5. **Block Modes**: Trace CBC/CTR encryption for 2-3 blocks

6. **CRT**: Solve system of congruences
7. **Miller-Rabin**: Test if number is prime
8. **HMAC**: Calculate MAC for given message

Theory Focus Areas

- **Security comparisons**: ECB vs CBC vs CTR
- **Attack scenarios**: When is each attack applicable?
- **Algorithm selection**: When to use RSA vs DHKE vs AES?
- **Mode selection**: When to use which block cipher mode?
- **Hash properties**: Collision resistance, one-way, avalanche

Common Mistakes to Avoid

- ✖ Forgetting $\mod n$ in calculations
- ✖ Confusing public/private keys in PKC
- ✖ Wrong order in DHKE (must use other's public key)
- ✖ Not initializing IV in CBC mode
- ✖ Mixing up encryption/decryption formulas in modes

QUICK CALCULATION TIPS

Fast Modular Exponentiation

For $a^b \mod n$:

1. Express b in binary
2. Calculate $a^{2^i} \mod n$ for each bit
3. Multiply corresponding values for '1' bits

Example: $7^{13} \mod 20$

- $13 = 8 + 4 + 1$
- $7^1 = 7, 7^2 = 49 \mod 20 = 9, 7^4 = 81 \mod 20 = 1, 7^8 = 1$
- Result: $7 \times 9 \times 1 \times 1 = 63 \mod 20 = 3$

Finding Multiplicative Inverse

Use EEA to find $d = MI(e) \mod n$:

1. Apply EEA to get $GCD(e, n) = 1 = ex + ny$
2. Then $d = x \mod n$

FINAL CHECKLIST

Before Exam:

- ✓ Practice RSA key generation (3-5 problems)
- ✓ Practice DHKE shared secret calculation
- ✓ Understand all 5 block cipher modes thoroughly
- ✓ Review attack types and their countermeasures
- ✓ Memorize key formulas (RSA, DHKE, HMAC)
- ✓ Practice modular arithmetic problems
- ✓ Understand security trade-offs

During Exam:

- Read questions completely
- Show all calculation steps

- Don't forget mod operations
- Check if answer makes sense (e.g., result should be < n)
- Manage time (don't spend too long on one question)

Good Luck!

[1] [2] [3]

⁂

1. poc_merged-notes_endsem.pdf
2. poc_merged-notes_endsem.pdf
3. poc_merged-notes_endsem.pdf